

# Reviewer Pack — Minimal Rank of p-Adic Divergence for Boolean Functions vs R...

Entry 9b02e8e9a110 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 19:05:28 UTC

## Minimal Rank of p-Adic Divergence for Boolean Functions vs Randomized Communication Complexity for Disjointness

Entry ID: 9b02e8e9a110 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 19:05:28 UTC

### 1. Conjecture

**Field A** (mathematical branch): p-adic Number Theory (Divergence) **Field B** (complexity object): Communication Complexity: Disjointness

#### Statement:

[‘For any boolean function  $f$ , the p-adic divergence  $D_p(f)$  is lower-bounded by the randomized communication complexity  $C_{\text{DISJ}}(n)$ , such that  $D_p(f) = \Omega(C_{\text{DISJ}}(n))$ .’, ‘Here,  $D_p(f)$  denotes the p-adic divergence of the function  $f$  computed over a finite field of size  $p$ , and  $C_{\text{DISJ}}(n)$  is the randomized communication complexity for the disjointness problem on  $n$  bits, which is known to be  $\Omega(n)$ .’, ‘Additionally, if there exists a boolean function  $f$  with  $D_p(f) \leq \varepsilon(C_{\text{DISJ}}(n))$ , then there exists an efficient algorithm to distinguish between the input strings and their complements with communication complexity  $O(1)$ ’]

#### Rationale (proposer’s reasoning):

[‘p-adic divergence is a measure of information loss, which could be related to the amount of communication needed to transmit information. Since communication complexity for disjointness is a measure of the minimum number of bits that must be exchanged to determine if two sets are disjoint, there might exist an intrinsic connection between these concepts.’, ‘This conjecture suggests that understanding the structure of boolean functions in terms of p-adic divergence could provide insights into the complexity of information transmission, potentially leading to new communication protocols with improved efficiency.’]

**Taxonomy category:** COMM\_DISJ (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 65a28d2857bc7fae

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

The conjecture is supported if the Pearson's correlation coefficient ( $r$ ) between p-adic divergence  $D_p(f)$  and randomized communication complexity  $C_{\text{DISJ}}(n)$  across at least 30 seeds is  $\geq 0.8$ , AND no seed produces a metric value greater than 10.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 1.00       | SAFE             | SAFE             |

### 4. Novelty audit

**Verdict:** NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "p-adic divergence" AND "randomized communication complexity" AND disjointness" - " $D_p(f)$ " AND  $C_{\text{DISJ}}(n)$  AND boolean function" - "efficient algorithm" AND p-adic number theory AND communication complexity

**Top relevant hits considered:** - [<http://arxiv.org/abs/1502.00357v2>] On Restricting No-Junta Boolean Function and Degree Lower Bounds by Polynomial Method - [<http://arxiv.org/abs/2302.02473v4>] Level-p-complexity of Boolean Functions using Thinning, Memoization, and Polynomials - [<http://arxiv.org/abs/math/0512459v1>] A p-adic Height Function Of Cryptanalytic Significance - [<http://arxiv.org/abs/1406.1273v2>] On The Communication Complexity of Finding an (Approximate) Stable Marriage - [<http://arxiv.org/abs/1310.4421v1>] Efficient computation of Rankin p-adic L-functions - [s2:10.1017/bsl.2018.13] 2017 EUROPEAN SUMMER MEETING OF THE ASSOCIATION FOR SYMBOLIC LOGIC LOGIC COLLOQUIUM '17 Stockholm, Sweden August 14-20,

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=0, elapsed=0.4s

#### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def p_adic_divergence(f, p):
    n = int(math.log2(len(f)))
    count = sum(1 for x in range(2**n) if f[x] != f[0])
    return Fraction(count, 2**n)

def communication_complexity_disjointness(n):
    return n

def run_trial(seed: int) -> dict:
```

```

random.seed(seed)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_boolean_function(n)
    D_p_f = p_adic_divergence(f, 2) # Using base 2 for simplicity
    C_DISJ_n = communication_complexity_disjointness(n)

    if D_p_f > 10 or C_DISJ_n > 10:
        return {
            "metric_name": "p-adic divergence vs. communication complexity",
            "metric_value": None,
            "instances_tested": n_values.count(n),
            "conjecture_holds": False,
            "counterexample": f"Metric value exceeded 10 for n={n}"
        }

    results.append((D_p_f, C_DISJ_n))

D_p_mean = sum(D_p for D_p, _ in results) / len(results)
C_DISJ_mean = sum(C_DISJ for _, C_DISJ in results) / len(results)

correlation_coefficient = 0
if D_p_mean != 0 and C_DISJ_mean != 0:
    numerator = sum((D_p - D_p_mean) * (C_DISJ - C_DISJ_mean) for D_p, C_DISJ in results)
    denominator = math.sqrt(sum((D_p - D_p_mean)**2 for D_p, _ in results)) * math.sqrt(sum((C_DISJ - C_DISJ_mean)**2 for _, C_DISJ in results))
    correlation_coefficient = numerator / denominator

conjecture_holds = correlation_coefficient >= 0.8
counterexample = "" if conjecture_holds else "Correlation coefficient < 0.8"

return {
    "metric_name": "p-adic divergence vs. communication complexity",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + [79]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}}, **trial_result}}")
        results.append(trial_result)

    D_p_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    C_DISJ_values = [r["instances_tested"] for r in results if r["instances_tested"] > 0]

    if len(D_p_values) == 0:

```

```

print("RESULT: INCONCLUSIVE no valid data")
else:
    mean_D_p = sum(D_p_values) / len(D_p_values)
    std_D_p = math.sqrt(sum((x - mean_D_p)**2 for x in D_p_values) / len(D_p_values))

    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_D_p} std={std_D_p} support_fraction={support_fraction}")
    elif any(r["metric_value"] > 10 for r in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if result["metric_value"] > 10)
        print(f"RESULT: FALSIFIED counterexample='Metric value exceeded 10' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE support_fraction < 0.8")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

TRIAL: {'seed': 11, **trial_result}
TRIAL: {'seed': 23, **trial_result}
TRIAL: {'seed': 37, **trial_result}
TRIAL: {'seed': 53, **trial_result}
TRIAL: {'seed': 71, **trial_result}
TRIAL: {'seed': 89, **trial_result}
TRIAL: {'seed': 103, **trial_result}
TRIAL: {'seed': 127, **trial_result}
TRIAL: {'seed': 149, **trial_result}
TRIAL: {'seed': 167, **trial_result}
TRIAL: {'seed': 191, **trial_result}
TRIAL: {'seed': 211, **trial_result}
TRIAL: {'seed': 233, **trial_result}
TRIAL: {'seed': 257, **trial_result}
TRIAL: {'seed': 277, **trial_result}
TRIAL: {'seed': 311, **trial_result}
TRIAL: {'seed': 347, **trial_result}
TRIAL: {'seed': 389, **trial_result}
TRIAL: {'seed': 421, **trial_result}
TRIAL: {'seed': 463, **trial_result}
TRIAL: {'seed': 503, **trial_result}
TRIAL: {'seed': 547, **trial_result}
TRIAL: {'seed': 593, **trial_result}
TRIAL: {'seed': 631, **trial_result}
TRIAL: {'seed': 677, **trial_result}
TRIAL: {'seed': 727, **trial_result}
TRIAL: {'seed': 773, **trial_result}
TRIAL: {'seed': 821, **trial_result}
TRIAL: {'seed': 877, **trial_result}
TRIAL: {'seed': 929, **trial_result}
RESULT: INCONCLUSIVE no valid data

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

## Critic reasoning:

The empirical test reports an inconclusive result with ‘no valid data’. This suggests that the test may not have been properly conducted or that the data collected is insufficient to draw a conclusion. The small sample size of  $n \leq 15$  could be too small to capture the behavior of the conjecture, and it does not rule out the possibility of metric saturation or selection bias.

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

### Reasoning:

The empirical test reported an inconclusive result with ‘no valid data’, suggesting that the test may not have been properly conducted or that the data collected is insufficient to draw a conclusion. The small sample size of  $n \leq 15$  could be too small to capture the behavior of the conjecture, and it does not rule out the possibility of metric saturation or selection bias. | next: Increase the number of seeds used in the test to at least 30 and ensure proper data collection methods are employed.

## 11. Audit log (LLM calls)

**Total LLM calls:** 12

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 10252        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 10667        |
| 3  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11194        |
| 4  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 5427         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4795         |
| 6  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 6495         |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13781        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11042        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10453        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11817        |
| 11 | critic          | ollama_remote | glm4:latest      | 0         | 0   | 18256        |
| 12 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 6337         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120516 ms total latency. Provider mix: {'ollama\_remote': 12}

(full prompt+response transcripts available in `research/audit/9b02e8e9a110.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9b02e8e9a110.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/9b02e8e9a110.tar.gz` (if generated)